

Automation First Project In Python

Description: Making a file management system that automatically creates folder structure, and move the image, video and document files to the respective folder in **Python**.

Look at this code written in simple way:

```
"""
Download Folder Automation System
-----

This program:
1. Creates a backup of the target folder.
2. Compresses the backup into a ZIP archive.
3. Organizes files into categories based on file extensions.
4. Displays an automation summary.
"""

from pathlib import Path
import shutil

# =====
# File Categories
# =====

FILE_TYPES = {
    "Images": [".jpg", ".jpeg", ".png", ".gif"],
    "Documents": [".pdf", ".txt", ".docx"],
    "Videos": [".mp4", ".mkv", ".avi"],
    "Audio": [".mp3", ".wav", ".aac"],
    "Archives": [".zip", ".rar", ".tar"],
    "Code": [".py", ".cpp", ".c", ".java", ".html", ".css", ".js"],
    "Data": [".csv", ".xlsx", ".json"],
}

# =====
# Create Backup
# =====

def create_backup(folder_path):
    """
    Creates a complete backup of the folder
    and compresses it into a ZIP archive.
    """

    backup_folder = folder_path.parent / f"{folder_path.name}_Backup"

    try:
```

```

# Remove old backup if it exists
if backup_folder.exists():
    shutil.rmtree(backup_folder)

# Copy entire folder
shutil.copytree(folder_path, backup_folder)

# Create ZIP archive
shutil.make_archive(str(backup_folder), "zip", backup_folder)

return True

except Exception as e:
    print("Backup Error:", e)
    return False

# =====
# Get Category
# =====

def get_category(extension):
    """
    Returns category name according to file extension.
    """

    extension = extension.lower()

    for category, extensions in FILE_TYPES.items():
        if extension in extensions:
            return category

    return "Others"

# =====
# Organize Files
# =====

def organize_files(folder_path):
    """
    Organizes all files into categorized folders.
    """

    summary = {
        "Images": 0,
        "Documents": 0,
        "Videos": 0,
        "Audio": 0,
        "Archives": 0,
        "Code": 0,
    }

```

```

        "Data": 0,
        "Others": 0
    }

    try:

        for item in folder_path.iterdir():

            # Skip folders
            if item.is_dir():
                continue

            category = get_category(item.suffix)

            destination = folder_path / category
            destination.mkdir(exist_ok=True)

            shutil.move(str(item), str(destination / item.name))

            summary[category] += 1

    except Exception as e:
        print("Organization Error:", e)

    return summary

```

```

# =====
# Display Summary
# =====

```

```

def display_summary(summary, backup_status):

    total = sum(summary.values())

    print("\n===== AUTOMATION SUMMARY =====")
    print(f"Images moved: {summary['Images']}")
    print(f"Documents moved: {summary['Documents']}")
    print(f"Videos moved: {summary['Videos']}")
    print(f"Audio files moved: {summary['Audio']}")
    print(f"Archives moved: {summary['Archives']}")
    print(f"Code files moved: {summary['Code']}")
    print(f>Data files moved: {summary['Data']}")
    print(f"Other files moved: {summary['Others']}")
    print("-----")
    print(f"Total files organized: {total}")

    if backup_status:
        print("Backup created successfully.")
    else:
        print("Backup creation failed.")

```

```

    print("File organization completed successfully.")

# =====
# Main Function
# =====

def main():

    folder = input("Enter Download Folder Path: ").strip()

    download_folder = Path(folder)

    if not download_folder.exists():
        print("Folder does not exist.")
        return

    print("\nCreating backup...")
    backup_status = create_backup(download_folder)

    print("Organizing files...")
    summary = organize_files(download_folder)

    display_summary(summary, backup_status)

# =====
# Program Entry
# =====

if __name__ == "__main__":
    main()

```

If you want to deploy it with **Streamlit**, just install the library using: `pip install streamlit`, and here is the code:

```

import streamlit as st
from pathlib import Path
import shutil

st.set_page_config(
    page_title="Download Folder Automation",
    page_icon="📁",
    layout="centered"
)

# =====
# File Categories
# =====

```

```

FILE_TYPES = {
    "Images": [".jpg", ".jpeg", ".png", ".gif"],
    "Documents": [".pdf", ".txt", ".docx"],
    "Videos": [".mp4", ".mkv", ".avi"],
    "Audio": [".mp3", ".wav", ".aac"],
    "Archives": [".zip", ".rar", ".tar"],
    "Code": [".py", ".cpp", ".c", ".java", ".html", ".css", ".js"],
    "Data": [".csv", ".xlsx", ".json"],
}

# =====
# Backup Function
# =====

def create_backup(folder_path):
    backup_folder = folder_path.parent / f"{folder_path.name}_Backup"

    try:
        if backup_folder.exists():
            shutil.rmtree(backup_folder)

        shutil.copytree(folder_path, backup_folder)

        archive_path = shutil.make_archive(
            str(backup_folder),
            "zip",
            backup_folder
        )

        return True, archive_path

    except Exception as e:
        return False, str(e)

# =====
# Category Function
# =====

def get_category(extension):
    extension = extension.lower()

    for category, extensions in FILE_TYPES.items():
        if extension in extensions:
            return category

    return "Others"

# =====
# Organize Files
# =====

```

```

def organize_files(folder_path):

    summary = {
        "Images": 0,
        "Documents": 0,
        "Videos": 0,
        "Audio": 0,
        "Archives": 0,
        "Code": 0,
        "Data": 0,
        "Others": 0
    }

    try:

        for item in folder_path.iterdir():

            if item.is_dir():
                continue

            category = get_category(item.suffix)

            destination = folder_path / category
            destination.mkdir(exist_ok=True)

            shutil.move(
                str(item),
                str(destination / item.name)
            )

            summary[category] += 1

        return summary

    except Exception as e:
        st.error(f"Error: {e}")
        return summary

# =====
# Summary
# =====

def show_summary(summary, backup_success):

    total = sum(summary.values())

    st.success("Automation Completed Successfully!")

    st.subheader("📊 Automation Summary")

```

```

st.table({
    "Category": [
        "Images",
        "Documents",
        "Videos",
        "Audio",
        "Archives",
        "Code",
        "Data",
        "Others",
        "Total"
    ],
    "Files": [
        summary["Images"],
        summary["Documents"],
        summary["Videos"],
        summary["Audio"],
        summary["Archives"],
        summary["Code"],
        summary["Data"],
        summary["Others"],
        total
    ]
})

```

```

if backup_success:
    st.success("✅ Backup created successfully.")
else:
    st.error("❌ Backup could not be created.")

```

```

# =====
# Main
# =====

```

```

def main():

    st.title("📁 Download Folder Automation System")

```

```

    st.write(
        """

```

Organize files automatically into categories.

****Categories****

- Images
- Documents
- Videos
- Audio
- Archives
- Code
- Data
- Others

```

"""
)

folder = st.text_input(
    "Enter Folder Path",
    placeholder=r"C:\Users\Student\Downloads"
)

if st.button("🚀 Organize Files"):

    if not folder:
        st.warning("Please enter a folder path.")
        return

    folder_path = Path(folder)

    if not folder_path.exists():
        st.error("Folder does not exist.")
        return

    with st.spinner("Creating Backup..."):
        backup_status, backup = create_backup(folder_path)

    with st.spinner("Organizing Files..."):
        summary = organize_files(folder_path)

    show_summary(summary, backup_status)

    if backup_status:
        with open(backup, "rb") as file:
            st.download_button(
                label="⬇ Download Backup ZIP",
                data=file,
                file_name=Path(backup).name,
                mime="application/zip"
            )

if __name__ == "__main__":
    main()

```

To run the code from `streamlit` you have to run that command in terminal:

```
streamlit run [file_name.py]
```

Example: `streamlit run app.py`